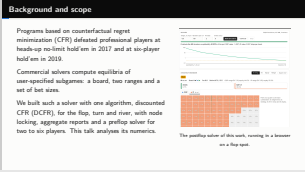


Timing for 60 minutes. Problem and equilibria 8 · CFR and DCFR 8 · Vectorized traversal and leaf evaluation 10 · Cost, memory, precision and parallelism 10 · Convergence and validation 6 · Live demonstration 10 · Extensions and next steps 8.

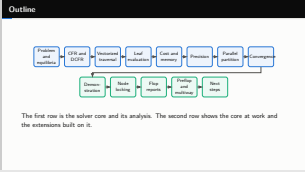
Opening line: "A poker solver is one numerical method, discounted counterfactual regret minimisation, applied to a structured bilinear saddle-point problem. This talk is about what makes that method fast and accurate in practice: how each iteration is organised, what it costs, how precise it needs to be and how it is split across processors."

Everything shown runs in the browser on the site in the repository, and every measured number comes from the scripts in `web/experiments`. The machine for the measurements is a two-core cloud container, so absolute times are pessimistic.

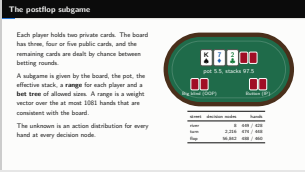


Libratus and DeepStack (2017) and Pluribus (2019) are the milestones. Their inner loop is a member of the CFR family. Commercial tools such as PioSolver and MonkerSolver are what players use; the feature list of this project follows them: flop, turn and river solving, arbitrary bet trees, rake, node locking, aggregate flop reports, preflop and multiway solving.

The scope is deliberately narrow on the algorithm side. There is a single method, DCFR, and the talk is about its numerical analysis: complexity per iteration, memory, floating-point precision, parallel decomposition and measured convergence. New algorithms are in the next-steps slide.

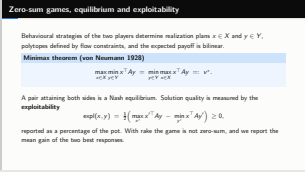


The first row is the numerical analysis. The second row is the live system and what is built on the same core: locking, reports and the preflop solver. Keep this slide short.



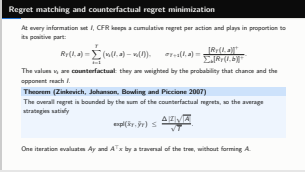
The running example is the most common spot in online poker: the button opens, the big blind calls, and the flop is K♠ 7♢ 2♣. The big blind has 488 hands and the button 460 that are consistent with the board.

The table shows the sizes of the trees that this talk measures. A river spot is a few dozen decision nodes; a flop spot with one bet size per street has tens of thousands of decision nodes, because every turn card and every river card opens a new copy of the later streets. The unknowns are one probability per hand, action and decision node.



This is a convex-concave saddle-point problem with bilinear coupling, and the exploitability is the primal-dual gap. Commercial solvers stop between 0.1% and 0.5% of the pot; the defaults of our site are 0.3%.

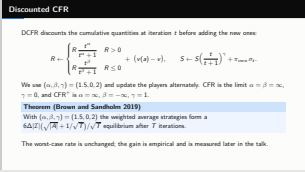
The sequence-form linear program of Koller, Megiddo and von Stengel solves this exactly and is polynomial in the tree, but the payoff matrix has a dense hand-by-hand block at every leaf. A flop tree has hundreds of thousands of leaves with blocks of size about 500×500, so the matrix cannot be stored. Iterative methods that only need products with A and $A^\top$ are the only option, and the next slides show that these products never need A either.



Regret matching (Hart and Mas-Colell 2000) guarantees that the maximal regret grows like $\sqrt{T}$. The proof is Blackwell approachability: the instantaneous regret is orthogonal to the current positive regret vector, so the squared norm of the positive regrets grows at most linearly.

CFR applies regret matching separately at each information set, with the counterfactual value as payoff. The decomposition theorem says the global regret, over an exponentially large strategy space, is at most the sum of these local regrets. The folk theorem turns average regret ε for both players into a 2ε equilibrium of the averages.

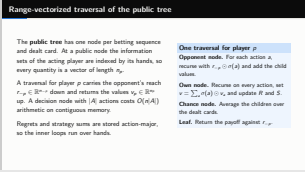
Δ is the payoff range, $|\mathcal{I}|$ the number of information sets and $|A|$ the largest number of actions.



Interpretation for numerical analysts. Early iterates are poor, and uniform averaging keeps them forever. Discounting is a forgetting factor: positive regrets lose a fraction $1/(t^{1.5}+1)$, negative regrets are halved ($\beta = 0$ gives the factor $1/2$), and the average gives weight proportional to t^2 to iteration t.

Alternating updates are Gauss–Seidel in place of Jacobi: the second player already responds to the first player's new strategy.

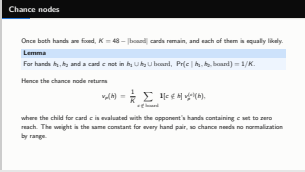
The theorem keeps the $T^{-1/2}$ bound. The practical rates are much better, and the convergence slide measures them on our spots.



This is the bridge between the game-theoretic formulas and the code. A naive implementation traverses the tree once per hand pair. The vectorized form traverses it once per player and iteration, and each node does vector arithmetic of length about 500.

The products Ay and $A^\top x$ of the previous slide are exactly the values returned to the root. The opponent's reach plays the role of y, and the payoff blocks are applied at the leaves without being stored. Own reach is needed only for the strategy sums.

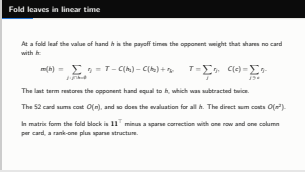
Implementation detail: all buffers for one depth are allocated once, so a traversal allocates no memory.



The point is that the conditional probability of the next card does not depend on the ranges, only on the four known private cards, which are excluded by the indicator and by zeroing the opponent's reach. If one used $1/(52 - |\text{board}|)$ instead, values would be biased for every hand.

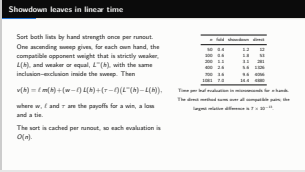
The lemma is checked numerically: on a turn game the solver agrees with an exact linear program built independently in Python with explicit chance nodes, to eight digits (validation slide).

Chance nodes multiply the tree. A flop has 49 turn cards and each turn 48 river cards, which is why flop spots are three orders of magnitude larger than river spots.



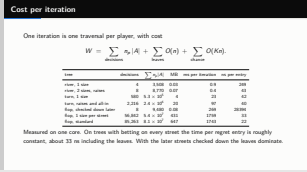
Inclusion–exclusion over the two cards of h. Two different two-card hands share at most one card, except the identical hand, which shares both and is counted in both $C(h_1)$ and $C(h_2)$. Adding it back gives the exact count.

This is the card-removal effect that poker players talk about, computed exactly at a cost of one pass over each list. The method is due to Johanson and coauthors (2011).



After sorting, the sign matrix of the showdown is lower triangular ones minus upper triangular ones, a semiseparable matrix, so its product with a vector is a prefix sum. Ties form blocks on the diagonal, handled by the two sums L and $L^{=}$. The compatibility mask is again handled by per-card sums that are updated inside the sweep.

The table shows the asymptotics. At 1081 hands the sweep is more than two hundred times faster than the direct sum, and the two agree to rounding. A flop iteration evaluates about seventy thousand showdown leaves, so this is the difference between seconds and hours.


Cost per iteration

One iteration is one traversal per player, with cost

$$W = \sum_{\text{decisions}} n_p |A| + \sum_{\text{leaves}} O(n) + \sum_{\text{chance}} O(Kn).$$

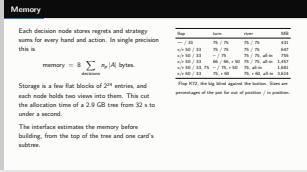
tree	decisions	$\sum n_p\|A\|$	MB	ms per iteration	ns per entry
river, 1 size	6	3,948	0.03	0.9	288
river, 2 sizes, raises	10	1.2×10^4	0.09	3.4	283
turn, 1 size	580	5.3×10^5	4	23	42
turn, raises and all-in	2,216	2.4×10^6	20	97	40
flop, checked down later	6	9,480	0.08	266	28396
flop, 1 size per street	56,842	5.4×10^7	431	1758	33
flop, standard	85,263	8.3×10^7	647	1743	22

Measured on one core. On trees with betting on every street the time per regret entry is roughly constant, about 33 ns including the leaves. With the later streets checked down the leaves dominate.

The work count $\sum n_p|A|$ is the number of regret entries, which is also what has to be read and written per player traversal. The constant of about thirty nanoseconds per entry includes the leaf work, which is about a third of the time on flop trees. Memory bandwidth, not arithmetic, is the limit on the large trees: each regret and strategy entry is read and written once per iteration.

Adding a bet size multiplies the tree, because it adds a branch at every node of the street and copies of all later streets below it.

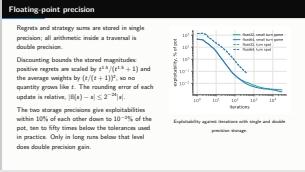
That is why commercial tools encourage few sizes on the flop.



Memory is the binding constraint in a browser. One typed array per node made the allocator the bottleneck. Flat blocks make allocation and garbage collection cheap and keep each node's data contiguous.

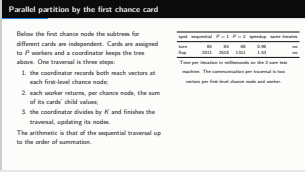
The estimate uses the fact that below a chance node the subtrees for different cards have the same shape; only the hands differ.

Building one card's subtree and multiplying by the number of cards is exact up to the small variation of hand counts.



Half the memory for the same practical result. The stall of single precision in long runs is visible on the small game, where it reaches 2.9×10^{-3}% of the pot after 30,000 iterations against 2.0×10^{-3}% in double precision. On the turn spot the two curves are indistinguishable over 400 iterations (0.071% and 0.076%).

The reason single precision suffices: the strategy is a ratio of sums, so relative errors of the sums enter the strategy as relative errors of size 2^{-24} per update. With uniform averaging the sums grow like t and each new term is relatively tiny, which is where single precision would lose information; the DCFR weights keep the newest terms relevant.


Parallel partition by the first chance card

Below the first chance node the subtrees for different cards are independent. Cards are assigned to P workers and a coordinator keeps the tree above. One traversal is three steps:

1. the coordinator records both reach vectors at each first-level chance node;
2. each worker returns, per chance node, the sum of its cards' child values;
3. the coordinator divides by K and finishes the traversal, updating its nodes.

The arithmetic is that of the sequential traversal up to the order of summation.

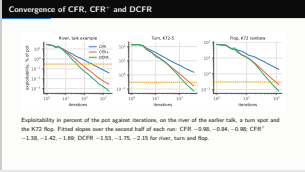
spot	sequential	$P=1$	$P=7$	speedup	same iterates
turn	65	84	60	0.96	no
flop	2011	2616	1321	1.53	no

Time per iteration in milliseconds on the 2-core test machine. The communication per traversal is two vectors per first-level chance node and earlier.

This is a domain decomposition of the tree. The interface between coordinator and workers is the set of first-level chance nodes, and the messages are vectors of length n, a few kilobytes, while the work below each card is millions of operations. The coordinator part is a tiny fraction of the work, so by Amdahl's law the speedup is limited by load balance among cards, about $\lceil K/P \rceil$ cards per worker.

Exactness was checked: with the same number of iterations the partitioned and the sequential solver give the same exploitability to all printed digits on river, turn and flop games, and the same strategies and values at nodes on both sides of the partition.

In the browser the workers are Web Workers, one per core, and the coordinator runs on the main thread.



All three methods use the same code and differ only in (α, β, γ). DCFR is the fastest on every street, and reaches the practical target of 0.3% of the pot in 75 iterations on the flop and 185 on the turn. The fitted slopes are far steeper than the worst-case $-1/2$: about -1 for CFR, -1.4 for CFR$^+$ and -1.5 or steeper for DCFR on these spots.

Explaining why the modern variants beat the worst-case bound by so much is an open problem. What the numbers say for practice: the cost to a fixed target is a few hundred iterations, independent of the street, so the cost per iteration from the previous slides is what decides the running time.

Validation

test	reference	result
River, talk example	the earlier independent solver	same strategies, exploitability 1.38×10^{-4} after 5000 iterations
Turn with chance nodes	exact sequence-form LP in Python	game value 24.92419856 against 24.92419883
Partitioned solving	sequential solver	identical iterates on river, turn and flop
Node locking	sequential solver with the same lock	identical values
Heads-up push or fold	linear program on the same tables	values agree to five digits
Leaf evaluation	direct $O(s^2)$ sums	agreement to 10^{-12}

Each component has an independent check. The most important is the turn test, because it checks chance weighting, card removal and showdown evaluation together against a linear program built from a separate game implementation. The difference of 10^{-6} in the value is the remaining exploitability of DCFR after 20,000 iterations, not a modelling error.

The preflop check is the same idea one level up: the heads-up push-or-fold game has an exact LP formulation, and DCFR matches it.

Demonstration



Strategy browser after a flop solve: the big blind's turn strategy, after the flop checks through.

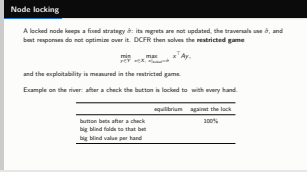
The site has five pages: postflop, preflop and multiway, flop reports, the river table of the earlier talk and a methods page. In the demonstration we

1. solve a flop and follow the exploitability,
2. browse strategies, EV and equity by turn card,
3. lock a node and re-solve,
4. solve a push-or-fold game and a six-player game.

Switch to the browser. The script with what to click and what to say is `app/DEMO_NOTES.md`. Budget ten minutes.

Short version. (1) Postflop page, example spot K72, one size per street. Build and solve; point at the memory estimate, the worker count and the exploitability curve on a log scale. (2) Walk: big blind check, button check, choose the five of hearts. Show the strategy grid, switch to EV and equity, point at one square to list combinations. (3) Lock: edit, choose the bet, paint the top pairs, lock, continue; show the opponent adapting. (4) Preflop page, heads-up push or fold at 10 bb: it converges in a second to the well-known 58% shove and 37% call. Then the six-max preset and the first-in table.

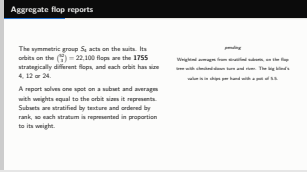
If time is short, skip the six-player game.



Locking answers the practical question: what is the best response to a known mistake, while every other node also adapts. It is a partial best response, more stable than a full best response because the rest of the tree is still solved jointly.

The example: a button who always bets can be called much more widely, and the big blind's value rises. The big blind's root strategy changes too, because checking has become more attractive.

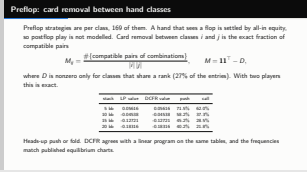
On flop spots a node below a chance card is locked for that card only, which matches the semantics of commercial tools.



By Burnside's lemma the number of orbits is the average number of fixed points over the 24 permutations of the suits, which gives 1755. The orbit sizes are 24 for rainbow boards, 12 for two-tone and paired boards and 4 for monotone and trips boards.

The report is a quadrature: a weighted average of a function of the flop. Stratification is the standard variance reduction. The table shows the estimates moving by less than the solver tolerance as the subset grows, so a 25 or 49 flop subset is enough for summary statistics.

The checked-down tree is the fast setting: it removes betting after the flop, so each flop takes seconds.



Preflop: card removal between hand classes

Preflop strategies are per class, 169 of them. A hand that sees a flop is settled by all-in equity, so postflop play is not modelled. Card removal between classes i and j is the exact fraction of compatible pairs

$$M_{ij} = \frac{\#\{\text{compatible pairs of combinations}\}}{|i|\,|j|}, \qquad M = \mathbf{1}\mathbf{1}^{\top} - D,$$

where D is nonzero only for classes that share a rank (27% of the entries). With two players this is exact.

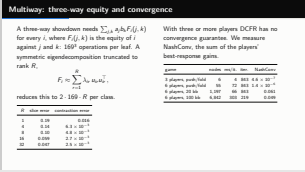
stack	LP value	DCFR value	push	call
5 bb	0.09616	0.09614	71.9%	62.0%
10 bb	-0.04538	-0.04533	58.2%	37.3%
15 bb	-0.13721	-0.13721	45.2%	29.9%
20 bb	-0.18316	-0.18314	40.2%	21.8%

Heads-up push or fold. DCFR agrees with a linear program on the same tables, and the frequencies match published equilibrium charts.

The equity tables are computed once offline: 169 by 169 class equities by Monte Carlo with 50,000 deals per pair, standard error about 0.2%, and exact compatible pair counts.

The product Mr costs $O(169^2)$ dense, but with D sparse it is a sum minus a sparse correction, about three times cheaper. Products are also cached per reach vector, because a vector formed at one node is shared by every leaf below it.

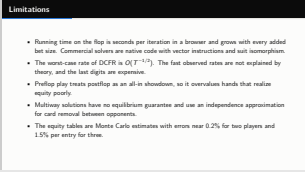
The LP for push or fold has 169 strategy variables and 169 constraints. DCFR reaches 10^{-7} bb in 3000 iterations, and the push and call frequencies at 10 bb, 58% and 37%, are the published Nash push-or-fold numbers.



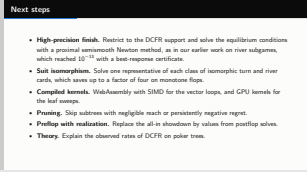
The tensor has 4.8 million entries, computed by Monte Carlo with 1000 deals per sorted triple. Its slices are far from low rank in the Frobenius sense, but the contraction with nonnegative range vectors is accurate: rank 16 gives a relative error below 0.3%, about the Monte Carlo error of the table itself.

Multiway convergence is an empirical statement. Push-or-fold games converge to 10^{-6} bb; the deep six-player trees reduce NashConv slowly and the result is an approximate equilibrium at best. This is consistent with the literature: Pluribus used CFR for six players without a guarantee, and it worked well in practice.

With three players, opponents' cards are treated as independent given the hero's hand, which ignores card removal between the opponents.

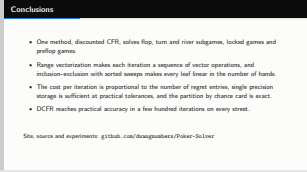


Be explicit. The browser engine is plain JavaScript; the next slide lists the engineering steps that close most of the gap to native solvers. The theoretical gap between observed and proven rates is a real open problem. The preflop limitation is the largest modelling simplification: commercial preflop solvers either solve the postflop as well (MonkerSolver) or use realization factors.



The first item is the research continuation. The earlier talk (`talk-newton.pdf`) showed that a proximal Newton method with interior-point crossover finishes a river subgame to machine precision from a DCFR start; the obstacle on larger trees was the flat solution set, handled by support restriction. Extending it to turn and flop trees needs the structured linear algebra of the leaves inside the Newton systems.

The other items are engineering with clear expected gains: isomorphism up to a factor of four on monotone boards, SIMD a factor of two to four, regret-based pruning large savings on deep trees.



Close by returning to the opening: the solver is a few lines of arithmetic, and its speed comes from the structure of the game, exploited at the level of vectors, leaves, memory and processors. Then take questions.

Appendix: the showdown sweep

Input: own list sorted by strength s, opponent list sorted by strength, opponent reach r.

1. Set $\mathrm{cum} \leftarrow 0$ and $C(c) \leftarrow 0$ for all cards.
2. For each block of own hands with equal strength s:
   - 2.1 add every opponent hand j with $s_j < s$ to cum and to C of its two cards;
   - 2.2 for each own hand h of the block set $L(h) \leftarrow \mathrm{cum} - C(h_1) - C(h_2)$;
   - 2.3 add every opponent hand with $s_j = s$, and set $L^{=}(h) \leftarrow \mathrm{cum} - C(h_1) - C(h_2) + r_j$.
3. After the sweep, $m(h)$ is the fold formula with the final sums.

Each opponent hand is added once and each own hand is read twice, so the cost is $O(n_1 + n_2)$.

The identical hand $\bar{h}$ cannot be strictly weaker than h, so it appears only in $L^{=}$. This is the reason for the correction term in step 2(c) only.

Appendix: counting flops with Burnside's lemma

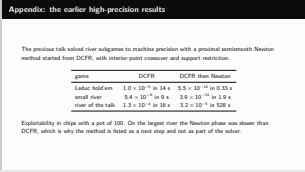
The number of orbits of S_4 on flops is the average number of flops fixed by a suit permutation g:

$$\#\text{orbits} = \frac{1}{|S_4|}\sum_{g\in S_4}|\mathrm{Fix}(g)| = \frac{22100 + 6\cdot 2938 + 8\cdot 299}{24} = 1755.$$

cycle type	count	flops fixed
identity	1	all $22\,100$
transposition $(a\,b)$	6	$\binom{13}{2}$ in the other two suits, plus a pair of equal rank in a and b with one more card: $2600 + 338$
3-cycle	8	$\binom{13}{3}$ in the fixed suit, plus one rank in the three cycled suits: $286 + 13$
double transposition, 4-cycle	3, 6	none

The code maps each flop to its smallest image under the 24 permutations and finds the same 1755 classes.

A flop has three cards, so a permutation that moves every suit cannot fix it unless its cards are permuted among themselves, which needs a cycle of length dividing the number of cards in the cycled suits. That rules out double transpositions and 4-cycles.


Appendix: the earlier high-precision results

The previous talk solved river subgames to machine precision with a proximal semismooth Newton method started from DCFR, with interior-point crossover and support restriction.

game	DCFR	DCFR then Newton
Leduc hold'em	1.0×10^{-4} in 14 s	5.5×10^{-16} in 0.33 s
small river	5.4×10^{-4} in 9 s	3.9×10^{-16} in 1.9 s
river of the talk	1.3×10^{-4} in 16 s	3.2×10^{-16} in 528 s

Exploitability in chips with a pot of 100. On the largest river the Newton phase was slower than DCFR, which is why the method is listed as a next step and not as part of the solver.

These are the numbers from `talk-newton.pdf`. The honest summary is that the finish is spectacular on small and medium games and not yet competitive on the largest river; the linear algebra of the dense leaf blocks inside the Newton systems is the bottleneck.